

CREApp

Manual de despliegue y operación

Configuración, publicación, recuperación y administración del servicio

Referencia técnica: rama **azure**, commit 7180921.

Revisión del contenido: 29 de septiembre de 2026.

Contenido

Propósito y alcance	1
1. Resumen de arquitectura y fuentes de verdad	1
2. Prerrequisitos y responsabilidades	3
3. Variables de entorno	3
4. Base de datos, TLS y backups	5
5. Staging local reproducible	6
6. Producción genérica: Ubuntu + Nginx + Gunicorn + PostgreSQL	7
7. Azure App Service con SPA y API en el mismo origen	10
8. Variante opcional: Azure Static Web Apps (SWA) + App Service split-origin	12
9. Verificación previa y posterior al despliegue	12
10. Operación continua, monitoreo y soporte	13
11. Diagnóstico rápido	13
12. Enlaces de referencia oficiales	14

Propósito y alcance

Este manual está dirigido a administradores de sistemas y profesionales de DevOps responsables de desplegar y operar CREApp. Describe la configuración, la conectividad con PostgreSQL, los procedimientos de publicación y recuperación, y las comprobaciones necesarias para validar un despliegue.

La referencia técnica es la rama **azure**, commit 7180921. Antes de operar, confirmar el commit desplegado y registrar los recursos, dominios, identidades, plan de servicio, DNS, secretos, copias de seguridad y responsables del entorno. La receta de máquina virtual requiere adaptar sus valores al servidor de destino.

Importante: este manual no sustituye a los procedimientos institucionales de seguridad, gestión de cambios, protección de datos ni recuperación ante desastres. Nunca copie secretos en tickets, logs, Git o esta documentación.

1. Resumen de arquitectura y fuentes de verdad

El backend necesita conectividad propia con PostgreSQL, tanto si la SPA se sirve desde App Service como desde Static Web Apps. La figura muestra las alternativas de acceso a la base; la elección debe corresponder a la red del entorno de destino.

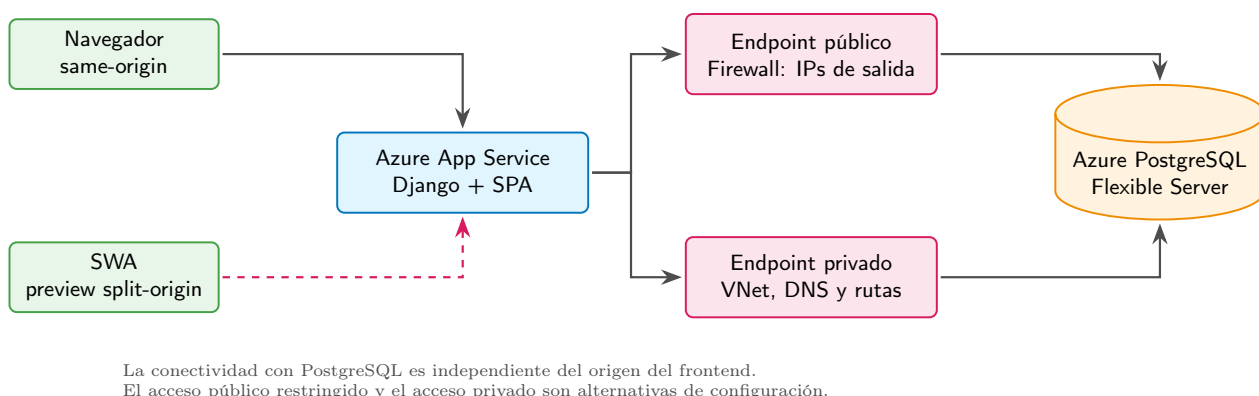


Figura 1. Arquitectura y topologías del despliegue

CREApp es una SPA React/TypeScript compilada con Vite y un backend Django que expone API REST mediante Django REST Framework, autenticación por sesión y PostgreSQL. En la variante same-origin, Django sirve la SPA y los estáticos desde el mismo origen; el cliente API usa `/api` por defecto y `frontend/src/services/api.ts` envía credenciales de navegador. Puntos de entrada comprobables:

Componente	Hecho del repositorio
Backend	<code>backend/config/settings.py</code> , apps <code>accounts</code> , <code>academics</code> , <code>planning</code> ; WSGI <code>config.wsgi:application</code>
API	<code>backend/config/urls.py</code> incluye <code>config.api_urls</code> bajo <code>/api/</code> ; DRF usa <code>SessionAuthentication</code> y permisos autenticados por defecto
Health	<code>GET /healthz</code> responde texto <code>ok</code> , no comprueba conexión de DB ni readiness completa
Admin	Prefijo actual <code>/api/sadmin-creapp-panel/</code> (no <code>/django-admin/</code>); los logins se redirigen allí
Estáticos compilados	<code>STATIC_URL=/api/static/</code> , <code>STATIC_ROOT=backend/staticfiles</code> ; <code>frontend/dist</code> es tanto directorio de plantillas como fuente de estáticos; middleware <code>WhiteNoise</code> está habilitado

Frontend	<code>frontend/package.json</code> ; Vite base lee <code>VITE_STATIC_BASE</code> o usa <code>/</code> ; <code>api.ts</code> usa <code>VITE_API_URL</code> o <code>/api</code>
Python	<code>runtime.txt</code> declara <code>python-3.13.0</code> ; <code>requirements.txt</code> fija dependencias Python (Django 6.0.2, DRF 3.16.1, Gunicorn 23.0.0, psycpg 3.2.13, WhiteNoise 6.11.0, entre otras)
Node	El workflow de App Service usa Node 22. Las dependencias están en <code>package.json</code> y la resolución en el lockfile; el host/runner local debe cumplir sus requisitos de motor.
DB de desarrollo	<code>backend/docker-compose.yaml</code> usa <code>postgres:17</code> , volumen Docker persistente y publica 5432 al host; es para desarrollo, no una receta segura para producción

Las versiones anteriores son una lectura del checkout, no una garantía de que un recurso Azure desplegado actualmente ejecute esas versiones. Consulte el commit desplegado, los logs de build y la configuración real antes de una operación.

Stack tecnológico y consecuencias operativas

La aplicación necesita un proceso Python que ejecute Django y acceso a PostgreSQL. La SPA ya compilada se ejecuta en el navegador. En el flujo del mismo origen, Node se utiliza en la etapa de build para generar los archivos que publica el backend.

Capa	Tecnología y responsabilidad operativa
Aplicación Python	Python 3.13; Django 6.0.2 y Django REST Framework 3.16.1. Instalar <code>requirements.txt</code> , aplicar las migraciones de la versión aprobada y reiniciar el proceso al cambiar su configuración. Las sesiones y CSRF deben conservar la configuración adecuada al origen público y al proxy.
Base de datos	PostgreSQL; psycpg 3.2.13 conecta desde Python. Verificar DNS, conectividad, TLS, permisos y copias de seguridad desde la red del backend. PostgreSQL 17 es la imagen de desarrollo de Compose, no una afirmación sobre el servidor Azure.
Servicio HTTP	Gunicorn 23.0.0 es la dependencia WSGI usada por la receta de VM; Nginx actúa como proxy en esa receta. En App Service, comprobar el comando de inicio efectivo y la terminación TLS del recurso. El workflow no demuestra qué proceso está ejecutándose en producción.
Interfaz compilada	React 19.2.3, TypeScript 5.9.3 y Vite 7.3.3, según el lockfile. Node 22 y <code>npm ci</code> preparan el build. <code>VITE_API_URL</code> y <code>VITE_STATIC_BASE</code> quedan incorporadas al artefacto: un cambio de estos valores exige recompilar y publicar la SPA.
Archivos estáticos	WhiteNoise 6.11.0 está configurado; <code>collectstatic</code> prepara el directorio y el manifiesto. Las rutas de Django incluyen una vista explícita para <code>/api/static/</code> . Validar las URLs reales de JS/CSS y su contenido después de publicar.
Configuración y CORS	<code>python-dotenv 1.2.1</code> carga <code>.env</code> ; <code>django-cors-headers 4.9.0</code> gestiona CORS. Inyectar los valores en el servicio y comprobar su efecto tras el reinicio. La elección de uno o dos orígenes cambia cookies y CORS, pero conserva la necesidad de conectividad del backend con PostgreSQL.
Publicación	GitHub Actions construye y publica los artefactos mediante los workflows del repositorio. Azure App Service aloja la aplicación; el workflow de Static Web Apps conserva las previews de PR. Antes de operar, verificar el workflow, el commit y el recurso destino.

Los paquetes de desarrollo y pruebas del frontend no constituyen servicios de producción. La explicación de los componentes de interfaz, formularios y pruebas está en la sección de stack tecnológico del [Manual de desarrollo](#).

Ramas pertinentes y divergencias de despliegue

- La configuración efectiva del checkout inspeccionado está en `backend/config/settings.py` y `backend/config/urls.py`: `/api/static/` y `admin /api/sadmin-creapp-panel/`. Partes de `docs/DEPLOY_AZURE`.

md todavía describen `/static/`, `/django-admin/`, otro FQDN y valores/IP de muestra. **No use esas rutas, nombres, IPs ni hostname como valores operativos sin verificarlos.**

- El workflow `.github/workflows/deploy.yml` compila con `VITE_API_URL=/api` y `VITE_STATIC_BASE=/api/static/`, consistente con el código actual. El base de Vite es una URL de recursos; el enrutamiento SPA lo hace Django.
- Referencia de producto/despliegue: **azure**. La configuración same-origin vigente en este checkout usa `/api/static/`, `admin /api/sadmin-creapp-panel/` y build frontend `VITE_API_URL=/api`, `VITE_STATIC_BASE=/api/static/`.
- **azure-same-origin** es una variante de despliegue, con `admin /sadmin-creapp-panel/`, estáticos `/static/` y build Vite `/static/`. En las referencias locales comparadas con **azure**, difiere en 53 rutas de archivo: le faltan 16 commits presentes en **azure** y tiene dos commits propios. No se asume que contenga las migraciones 0011/0012 ni las funciones actuales de producto. Antes de publicarla, verificar la lista de migraciones y compatibilidad con la base destino.
- **main** conserva una configuración previa con `admin /django-admin/` y estáticos `/static/`. Es relevante porque su push activa dos workflows de App Service descritos abajo. Está 23 commits detrás de **azure** en las referencias locales inspeccionadas; no tratarla como espejo sincronizado.
- La comparación usa branches y refs remotas locales disponibles al momento de revisión; no consulta el estado actual de GitHub ni del App Service. Otras ramas del repositorio no son una ruta operativa de publicación descrita en esta guía.
- Dos workflows apuntan al nombre de App Service **cre-app-api**: `deploy.yml` se activa por push a **main/azure** y por ejecución manual; `main_cre-app-api.yml`, por push a **main** y manualmente. Un push a **main** puede iniciar ambos, que construyen paquetes diferentes y usan autenticación distinta. No desplegar desde **main** hasta confirmar en Actions cuál workflow correrá y coordinar un único paquete final; el repositorio por sí solo no define cuál deploy termina último.
- `.github/workflows/azure-static-web-apps-lively-river-0fedd7a0f.yml` conserva **push** comentado y trigger de Pull Request en **azure**; publica previews de PR con API URL definida en el YAML. Un push a **azure** no publica SWA por ese workflow. No deducir que los secretos ni el recurso SWA siguen vigentes.

2. Prerrequisitos y responsabilidades

Antes de desplegar, asigne responsable de aplicación, base de datos, infraestructura/DNS/TLS, secretos y aprobación. Reúna y registre de forma segura: repositorio y commit aprobado; rama/entorno; hostname(s); estrategia de acceso a PostgreSQL; ventana y plan de rollback; límites de indisponibilidad; retención/ubicación de backups; contacto de guardia. No invente ni reutilice credenciales de ejemplo.

Requisitos técnicos mínimos observables/recomendados:

- Python 3.13 y herramientas de `venv/pip` según `runtime/workflow`; para VM, confirmar que **python3** apunta a un intérprete compatible antes de crear el `venv`. Usar una versión de PostgreSQL soportada por el proveedor; Compose **postgres:17** solo describe desarrollo.
- Node/npm compatible con el lockfile; workflow usa Node 22 y **npm ci** desde **frontend/**.
- **git**, acceso controlado al repo, HTTPS/TLS para todo tráfico público y un gestor seguro de secretos.
- PostgreSQL alcanzable desde el proceso de aplicación por red privada o reglas de firewall estrechas. Nunca exponga 5432 públicamente ni habilite una regla de acceso general a servicios Azure.
- En producción, cuenta de servicio sin login interactivo para la app, directorios con permisos mínimos, monitoreo, backups externos comprobados y procedimiento de recuperación probado.

3. Variables de entorno

El backend carga `backend/.env` mediante `python-dotenv` (`BASE_DIR/.env`, donde `BASE_DIR` es `backend/`). En servicios, configure variables en el mecanismo seguro del servicio en lugar de incluir ese archivo en el artefacto. `backend/.env.example` muestra solo parte de las variables. Las variables de lista se separan por comas. Para booleans el código acepta **true**, **1**, **yes**, **on** (sin distinción de mayúsculas); el resto se interpreta como falso.

Variable	Uso / default efectivo	Producción
----------	------------------------	------------

DEBUG	Default True si falta; peligroso	Obligatorio False
SECRET_KEY	Si falta y DEBUG es verdadero usa una clave insegura fija de desarrollo; con DEBUG falso arranca con error	Generar clave aleatoria fuerte; secreta, única por entorno; no rotar sin considerar sesiones firmadas
ALLOWED_HOSTS	Default <code>localhost,127.0.0.1</code>	Lista exacta de hostnames que sirven la app; sin esquema ni rutas
POSTGRES_DB	Sin default útil	Nombre de DB ya creada
POSTGRES_USER	Sin default útil	Usuario de aplicación con permisos necesarios, contraseña secreta y rotada según política
/ POSTGRES_PASSWORD		Host/FQDN privado o endpoint autorizado de DB
POSTGRES_HOST	Default <code>localhost</code>	Puerto del endpoint
POSTGRES_PORT	Default <code>5432</code>	Same-origin no suele necesitar CORS; split-origin: solo origen SPA exacto, con esquema
CORS_ALLOWED_ORIGINS	Default <code>http://localhost:5173,http://127.0.0.1:5173</code>	Orígenes HTTPS exactos que envían formularios/API; configurar el origen frontend real
CSRF_TRUSTED_ORIGINS	Mismo default local	No abrir CORS a todos los orígenes; credentialed requests requieren origen explícito
CORS_ALLOW_CREDENTIALS	En <code>settings.py</code> está siempre True , no se lee desde entorno	True tras HTTPS
CSRF_COOKIE_SECURE / SESSION_COOKIE_SECURE	Default False	
CSRF_COOKIE_SAMESITE / SESSION_COOKIE_SAMESITE	Default Lax	Same-origin: Lax ; split-origin cross-site puede requerir None y cookies Secure
SECURE_SSL_REDIRECT	Default False	True cuando proxy/servidor HTTPS está correctamente configurado
SECURE_HSTS_SECONDS	Default 0	Recomendación progresiva tras validar HTTPS y todos los subdominios; no activar HSTS/preload sin decisión de dominio
SECURE_HSTS_INCLUDE_SUBDOMAINS, SECURE_HSTS_PRELOAD	Default False	Solo habilitar tras confirmar implicaciones en todos los hosts/subdominios
SECURE_CONTENT_TYPE_NOSNIFF	Default True	Mantener habilitado
X_FRAME_OPTIONS	Default DENY	Mantener salvo requisito documentado
PROXY_ACCESS_SECRET	Vacío; middleware de acceso directo queda desactivado	Opcional, solo si arquitectura añade proxy de acceso que inyecta X-Proxy-Access-Secret ; secreto compartido. Configurar también bien excepciones
PROXY_ACCESS_EXEMPT_PATHS	<code>/healthz,/sadmin-creapp-panel/,/api/sadmin-creapp-panel/,/api/static/</code>	No cambiar sin entender <code>ProxyAccessMiddleware</code> ; omitir paths puede generar 403 confusos
VITE_API_URL	Vite no lo asigna; cliente API fallback / <code>api</code>	Build time: <code>/api</code> same-origin o URL completa del backend en split-origin
VITE_STATIC_BASE	Fallback Vite /	Build time: <code>/api/static/</code> para el workflow same-origin actual

Vite incorpora variables `VITE_*` al código público del navegador: **nunca colocar secretos ahí**. El workflow de App Service define sus dos variables en tiempo de build. Para split-origin, [.github/workflows/azure-static-](https://github.com/azure-static-workflows/azure-static-workflows)

`web-apps-lively-river-0fedd7a0f.yml` define URL de API del App Service como valor de build. No asuma que cambiar App Settings actualiza los valores Vite ya compilados: reconstruya y despliegue el frontend.

Generar una clave localmente, sin guardarla en el shell history si la política lo prohíbe:

```
python -c 'import secrets; print(secrets.token_urlsafe(50))'
```

4. Base de datos, TLS y backups

Creación y conectividad

1. Aprovechone PostgreSQL con versión/recursos según política de organización y cree explícitamente la base de datos definida por `POSTGRES_DB` (Django no la crea). Use un principal exclusivo de aplicación; evite un superusuario permanente para el servicio.
2. Mantenga el servidor en red privada cuando sea posible. Si el proveedor exige acceso público, permita solo origen(es) de salida estrictamente necesarios y TLS; jamás una regla amplia `0.0.0.0/0` ni “cualquier servicio Azure”. Restrinja también el firewall de red/NSG y `pg_hba` cuando aplique.
3. En App Service, acceso a PostgreSQL público requiere permitir **todas** las direcciones listadas en `Outbound IP addresses` y `Additional outbound IP addresses` de las propiedades del App Service, no una IP suelta. El tráfico puede salir por cualquiera y fallar intermitentemente si falta alguna. El conjunto puede cambiar al modificar plan/escala; audítelo tras cambios. Las IP son estado vivo y no constan aquí.
4. Con Private Endpoint/VNet: VNet Integration del App Service es una conexión saliente y requiere subnet dedicada compatible; no es el Private Endpoint. El Private Endpoint de PostgreSQL se ubica en la red/subnet según diseño del servicio. Configure rutas, NSG y DNS privado (resolución del FQDN al destino privado) y compruebe resolución/conectividad desde el contexto de App Service. No publique una DB privada como sustituto de DNS/VNet bien configurados. Los requisitos/tier de App Service para integración pueden generar costo; confirmar SKU vigente.
5. `settings.py` arma la conexión PostgreSQL con variables individuales, sin un parámetro SSL explícito. No afirmar que la aplicación impone `sslmode` ni validación de CA. Antes de operar con DB remota, verificar el requisito TLS del servidor y comprobar desde el runtime de aplicación qué modo/certificado negocia el driver. Los scripts de backup tampoco establecen SSL por sí solos; `PGSSLMODE` y certificados de libpq deben corresponder a la política de DB y comprobarse con el cliente PostgreSQL usado.

Backup / restore

`scripts/db-export.sh` lee `backend/.env` (o `ENV_FILE`) y crea un dump custom `pg_dump -F c`; el destino predefinido es la raíz del repo. El script interpreta líneas simples `KEY=VALUE`, comentarios y comillas externas; **no** evalúa sintaxis de shell ni equivale a `python-dotenv/systemd`. El archivo debe ser legible por el usuario que ejecuta el script. Para el archivo de producción `/etc/creapp/creapp.env`, revise que sus entradas sean compatibles con ese parser (sin `export`, expansiones, valores multilínea ni sintaxis shell) y use `ENV_FILE` explícitamente. Los dumps contienen datos sensibles: almacene fuera del repo, cifre en tránsito y reposo, limite acceso y establezca retención. Desde la raíz del checkout, use un directorio seguro fuera del repo y confirme que existe y tiene espacio/permisos:

```
cd /ruta/al/checkout/cre-app
ENV_FILE=/etc/creapp/creapp.env ./scripts/db-export.sh /ruta-segura/creapp-$(date
↪ +%Y%m%d_%H%M%S).dump
```

Los scripts `export/import` aplican credenciales de `.env` a herramientas PostgreSQL, pero no configuran SSL. Para DB remota con TLS, configure además las variables/archivos SSL apropiados para `pg_dump`, `psql`, `createdb` y `pg_restore` (por ejemplo `PGSSLMODE` según política) antes de usarlos. No ponga password en argumentos de línea de comandos.

`scripts/db-import.sh <dump>` consulta `postgres`, crea la DB si falta y ejecuta `pg_restore --clean --if-exists` sobre ella. No es restauración transaccional: ante errores pueden quedar objetos reemplazados y un resultado parcial antes de que el script informe fallo. El dump lógico no incluye roles globales, settings del servidor, archivos TLS ni DNS. Nunca uses este script como rollback improvisado en producción. Valida un restore del dump en una DB aislada, revisa el código de salida/salida de `pg_restore`, ownership y conectividad antes de considerar

recuperable el backup. Documenta RPO/RTO con los responsables; snapshots y backups administrados también deben probarse mediante restore.

5. Staging local reproducible

Desde la raíz del checkout. El ejemplo versionado del backend no contiene credenciales de desarrollo utilizables; editar una copia local de `backend/.env.example` a `backend/.env`, crear primero la DB y configurar valores locales:

```
python3 -m venv .venv
. .venv/bin/activate
python -m pip install -r requirements.txt
cp backend/.env.example backend/.env
```

Edite `backend/.env`: cree DB/usuario y reemplace los valores de ejemplo, incluidos `secret` y `password`. No comitee `.env`. Para PostgreSQL de desarrollo con Docker, Compose busca las variables del `.env` del directorio actual; ejecutar desde `backend/`:

```
cd backend
docker compose up -d
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
```

`runserver` solo para desarrollo. El cliente API usa `/api` relativo a Vite y el proxy lo envía a Django. No hace falta un archivo frontend local para esa modalidad. Si se accede directamente a Django, configura explícitamente `VITE_API_URL=http://localhost:8000/api`; `frontend/.env.example` no está versionado y no existe en un clon limpio. Luego:

```
cd frontend
npm ci
npm run dev
```

Al desarrollar con Vite, la configuración proxy reenvía `/api`, `/django-admin` y `/static` a `localhost`; las dos últimas rutas corresponden a la configuración de Vite, no a los endpoints productivos de `azure`. Para construir los assets same-origin de `azure`, usa:

```
cd frontend
VITE_API_URL=/api VITE_STATIC_BASE=/api/static/ npm run build
```

Desde `backend/`, y con el entorno local cargado, prueba `python manage.py collectstatic --noinput`. La base Vite y `STATIC_URL`/rutas Django deben pertenecer a la misma configuración de rama. Vite da prioridad a variables exportadas sobre archivos `.env`; revisa `frontend/dist/index.html` y las rutas de JS/CSS generadas para detectar destinos heredados.

En un build same-origin, `/api/programas/` no es una prueba de API válida: puede responder el `index.html` de la SPA con HTTP 200. Verifica el `Content-Type` JSON y una forma de respuesta API conocida. `/healthz` responde texto `ok`, pero no prueba la DB. Completa las pruebas de sesión/API contra staging antes de promover.

Comprobaciones recomendadas desde las carpetas indicadas:

```
# Desde raíz, dentro del venv Python y con backend/.env configurado.
# La DB indicada debe existir y estar accesible para estos comandos de estado de migración.
# makemigrations --check puede consultar historial/routers; migrate --check consulta DB.
cd backend
python manage.py check
python manage.py makemigrations --check --dry-run
python manage.py migrate --check
python manage.py test
```

```
# Desde frontend/  
npm ci  
npm run typecheck  
npm test  
npm run build
```

6. Producción genérica: Ubuntu + Nginx + Gunicorn + PostgreSQL

Es una receta de referencia para que el operador la adapte y revise, no archivos ya existentes en el repo. Use Ubuntu LTS aún soportado por Canonical, paquetes de seguridad actualizados, dominio y certificados vigentes. PostgreSQL debe estar en host/red privada, preferiblemente separado de la VM web. No ejecute aplicación como `root`, no sirva `runserver`, y no guarde secretos en el checkout.

Directorios y artefacto

Recomendación: release inmutable por commit en `/srv/creapp/releases/<commit>`, symlink `/srv/creapp/current`, entorno virtual propio dentro de cada release o compartido según operación. Antes de clonar, aprovisiona `/srv/creapp/releases` con propietario/grupo de despliegue y permisos que permitan escribir al operador de release; no haga el checkout como usuario del proceso web. Usuario/grupo de sistema `creapp`; archivos de código legibles pero no modificables por el proceso web, y los directorios que `collectstatic` deba escribir accesibles por `creapp`. Mantenga el entorno de producción fuera del repo en `/etc/creapp/creapp.env` con permisos mínimos (por ejemplo `root:creapp, 0640`); porque Django solo auto-lee `backend/.env`, systemd debe inyectar el entorno mediante `EnvironmentFile`. Asegure que el archivo no contiene sintaxis incompatible con `EnvironmentFile` de systemd; es distinto del formato `.env` de Django. Directorios de logs gestionados por journald. Si se usa uploads/media en futuro, definir un volumen persistente separado; no se documenta aquí almacenamiento media de aplicación.

En un checkout limpio, con `<URL_DEL_REPOSITORIO_AUTORIZADO>` sustituido por la URL Git real aprobada, `<COMMIT>` por el hash completo aprobado, y tras aprovisionar el directorio padre/permisos anteriores. Estos comandos clonan y luego fijan explícitamente el commit; compruebe que la salida de `rev-parse` coincide exactamente con el hash aprobado:

```
git clone <URL_DEL_REPOSITORIO_AUTORIZADO> /srv/creapp/releases/<COMMIT>  
cd /srv/creapp/releases/<COMMIT>  
git checkout --detach <COMMIT>  
git rev-parse HEAD  
python3 -m venv .env  
. .env/bin/activate  
python -m pip install --upgrade pip  
pip install -r requirements.txt  
cd frontend  
npm ci  
VITE_API_URL=/api VITE_STATIC_BASE=/api/static/ npm run build
```

Use valores reales en vez de los placeholders y no continúe si el hash mostrado no coincide. Provisionar `/etc/creapp/creapp.env` antes de ejecutar comprobaciones de producción: incluir como mínimo `DEBUG=False`, `SECRET_KEY`, `ALLOWED_HOSTS`, origins y credenciales DB; permisos mínimos y DB accesible. No `source` ese archivo en una shell. Inyecte el entorno por systemd con `EnvironmentFile` tanto para comprobaciones como para `collectstatic`/migraciones. `check --deploy` requiere settings productivos; `collectstatic` también necesita el entorno/paths correctos y un `frontend/dist` ya construido. Nunca suprima la salida ni use `|| true` para convertir errores en éxito.

Una forma de ejecutar un comando puntual con las propiedades systemd (systemd debe estar disponible y la invocación autorizada) es `systemd-run --wait --collect --pipe --property=User=creapp --property=Group=creapp --property=WorkingDirectory=/srv/creapp/releases/<COMMIT>/backend --property=EnvironmentFile=/etc/creapp/creapp.env /srv/creapp/releases/<COMMIT>/.env/bin/python manage.py check --deploy`. Repita sustituyendo el comando final por `collectstatic --noinput` o, en la

ventana aprobada, `migrate --noinput`. Antes de `collectstatic`, ajuste propietario/permisos de la carpeta `backend/staticfiles` para permitir escritura al usuario `creapp`; valide resultado. Si su `systemd` no admite esta invocación o la política impide `systemd-run`, use un mecanismo de ejecución equivalente que cargue variables de forma segura; no ejecute el comando sin `env` de producción ni haga `source` del archivo.

Entorno y migración segura

En `/etc/creapp/creapp.env` establezca como mínimo `DEBUG=False`, clave nueva, `ALLOWED_HOSTS` hostname real, DB vars correctas, origins adecuados y flags HTTPS seguros. Use valores sin `export`; evite espacios/comillas ambiguos en valores parseados por `python-dotenv`. Guarde `POSTGRES_PASSWORD` con permisos restrictivos. Proteja y haga backup antes de migrar.

Las migraciones son cambios de esquema potencialmente no reversibles. Despliegue en este orden: revisar migraciones/compatibilidad; backup verificable; preparar release y estáticos; aplicar migraciones con la nueva versión antes de cambiar tráfico si son compatibles hacia atrás; cambiar symlink/reiniciar; validar y monitorear. Para migraciones que rompen compatibilidad, use expansión/contracción en releases separadas y plan de downtime. Nunca automatice rollback DB sin conocer operaciones reversibles.

Las migraciones también deben ejecutarse con `EnvironmentFile` sin hacer `source` de secretos; use el patrón `systemd-run` anterior con `migrate --noinput`, en el release y ventana aprobados.

Gunicorn y systemd (plantilla)

Crear `/etc/systemd/system/creapp.service` con usuario/grupo y paths ajustados a despliegue aprobado:

```
[Unit]
Description=CREApp Django application
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=creapp
Group=creapp
WorkingDirectory=/srv/creapp/current/backend
EnvironmentFile=/etc/creapp/creapp.env
ExecStart=/srv/creapp/current/.venv/bin/gunicorn --workers 3 --bind 127.0.0.1:8000
↳ --access-logfile - --error-logfile - config.wsgi:application
Restart=on-failure
RestartSec=5
UMask=0027
NoNewPrivileges=true
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

El número de workers es recomendación ilustrativa, dimensionar según CPU/memoria/carga. App Service usa en un documento histórico timeout 600s; no copie ese valor automáticamente en VM. Si las peticiones legítimas largas lo requieren, revisar proxy timeout/capacidad. Gunicorn solo escucha loopback, no exponga 8000.

```
sudo systemctl daemon-reload
sudo systemctl enable --now creapp
sudo systemctl status creapp --no-pager
sudo journalctl -u creapp -n 100 --no-pager
```

Nginx, HTTPS y estáticos

El backend tiene una ruta catch-all SPA y una ruta explícita `/api/static/` que ejecuta `django.views.static.serve` contra `STATIC_ROOT`. `WhiteNoiseMiddleware` está instalado, pero esa ruta explícita no debe atribuirse automáticamente a `WhiteNoise`. Para la plantilla VM, Nginx puede proxyar las rutas a Gunicorn, o servir `/api/static/` desde el `STATIC_ROOT` del release con un alias verificado. Si Nginx sirve estáticos directo, confirmar `collectstatic`, permisos y ruta del release tras cada cambio. No usar `/static/` sin alinear settings, rutas, build y rama.

Fragmento de server HTTPS (certificados gestionados externamente por ACME/organización; no son valores reales):

```
server {
    listen 80;
    server_name <FQDN_REAL>;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name <FQDN_REAL>;
    ssl_certificate <RUTA_CERTIFICADO>;
    ssl_certificate_key <RUTA_CLAVE_PRIVADA>;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_redirect off;
    }
}
```

`SECURE_PROXY_SSL_HEADER=('HTTP_X_FORWARDED_PROTO','https')` está configurado en Django. Confíe en ese header solo si Nginx/proxy de confianza sobrescribe el valor recibido del cliente; no permita que cliente directo falsifique HTTPS. Django tiene `SECURE_SSL_REDIRECT` desactivado por defecto; actívelo (`True`) solo cuando HTTPS y los headers de proxy estén comprobados. Use `ALLOWED_HOSTS` con el dominio atendido. El firewall host/nube abre 80/443 y SSH administrativo restringido; 8000/5432 no deben ser públicos.

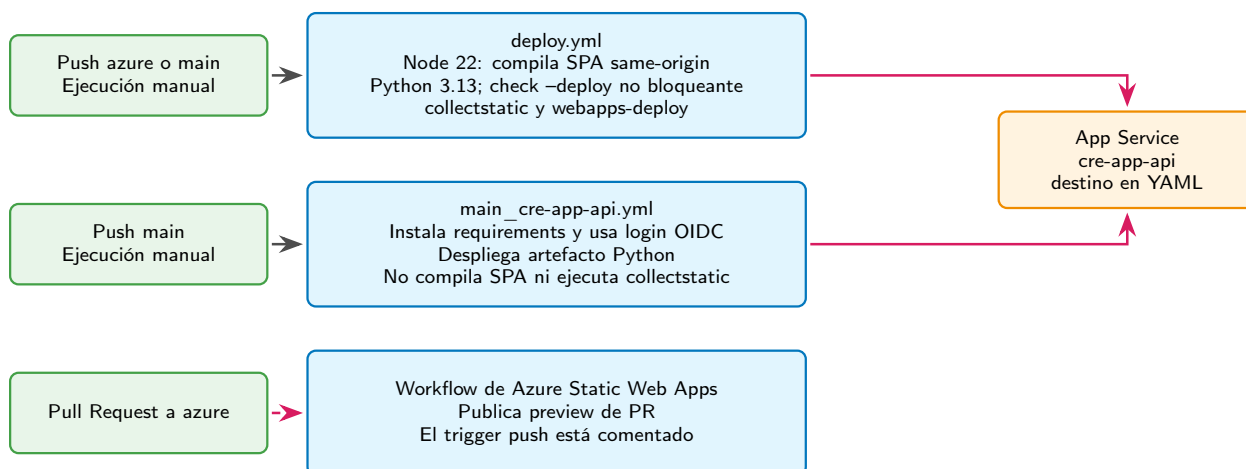
Release, verificación y rollback en VM

1. Construir nuevo release fuera del **current**; instalar, compilar, `check --deploy`, `collectstatic`; backup DB.
2. Determinar compatibilidad de migraciones. Aplicar con ventana si corresponde. No iniciar dos releases que migren simultáneamente.
3. Cambiar symlink atómicamente, reiniciar servicio; verificar `/healthz`, portada SPA, recursos JS/CSS, API, login/sesión/admin y logs.
4. Ante fallo de aplicación, volver symlink al release anterior y reiniciar **solo si el esquema aún es compatible**. Restaurar DB requiere decisión explícita: restaure backup probado, acepta pérdida de cambios desde punto de backup, y realiza mantenimiento. No ejecutar `db-import.sh` como rollback improvisado.

Ejemplo de cambio de enlace, ejecutar con privilegio y validar el destino antes:

```
ln -sf /srv/creapp/releases/<COMMIT_NUEVO> /srv/creapp/current.new
mv -Tf /srv/creapp/current.new /srv/creapp/current
sudo systemctl restart creapp
```

7. Azure App Service con SPA y API en el mismo origen



Un push a main puede iniciar ambos despliegues hacia el mismo App Service.
Ninguno aplica migraciones; deploy.yml tampoco ejecuta tests y permite continuar si falla check -deploy.
El commit activo, secretos, OIDC y estado del recurso se verifican en GitHub Actions y Azure Portal.

Figura 2. Flujos GitHub Actions del checkout azure

El despliegue same-origin sirve SPA y API desde App Service. `docs/DEPLOY_AZURE.md` da contexto histórico, pero sus hostnames, regiones, IPs, rutas y tier deben confirmarse. Azure CLI/Portal labels pueden cambiar; siga documentación Microsoft oficial enlazada abajo y valide estado vivo.

Preparación previa

1. Confirmar suscripción, grupo de recursos, App Service Linux/Python compatible con runtime 3.13, app **cre-app-api** (el nombre aparece en ambos workflows, no prueba estado vivo), plan/SKU y hostname HTTPS. Confirmar que Python 3.13 está soportado por la región/SKU/runtime actual.
2. Aprovisionar PostgreSQL Flexible Server y DB; decidir acceso privado/VNet o público restringido. Definir certificados/TLS y owner del backup. Probar desde la app conectividad y TLS, no desde laptop solamente.
3. Crear App Settings conforme a la matriz; valores críticos: `DEBUG=False`, `SECRET_KEY`, `ALLOWED_HOSTS` con hostname real sin esquema, `CSRF_TRUSTED_ORIGINS=https://<host>`, variables DB, flags Secure de cookies si todo tráfico está en HTTPS y `SECURE_SSL_REDIRECT=True` tras verificar TLS de proxy. Same-origin no necesita CORS salvo integración cross-origin. Microsoft indica que editar App Settings reinicia App Service; la nota operativa del repositorio requiere además usar Restart manual desde Portal al cambiar variables Django y comprobar la configuración efectiva del proceso. Reconocer ese reinicio evita asumir que la modificación quedó aplicada.
4. Configurar el startup command acorde al checkout verificado. `docs/DEPLOY_AZURE.md` propone `gunicorn --bind=0.0.0.0:8000 --timeout 600 --chdir backend config.wsgi:application --access-logfile '-' --error-logfile '-'`; confirme ruta/build Oryx y runtime vigentes antes de copiarlo. No inventar timeout ni cambiar workers sin carga observada.
5. GitHub: inspeccione Actions y confirme qué workflow se activará en la rama objetivo. En el estado checked-in, `deploy.yml` cubre `main,azure`; el workflow `main_cre-app-api.yml` también cubre `main`; ambos apuntan a `cre-app-api`. Ejecute despliegues solo con aprobación y tras resolver colisión. Secrets/auth reales no se conocen; no copiar secretos aquí. El primero requiere `AZURE_WEBAPP_PUBLISH_PROFILE`; el otro referencia secrets OIDC con nombres generados. Revisar permisos, expiración/rotación y destino.

Build y despliegue

`.github/workflows/deploy.yml` se activa al hacer push en `main` o `azure`, y manualmente en una ref elegida en GitHub. Construye frontend con Node 22, `npm ci, /api, /api/static/`; instala requirements con Python 3.13, ejecuta `check --deploy 2>&1 || true, collectstatic` y despliega el paquete a `cre-app-api` mediante `azure/webapps-deploy@v3`. El `|| true` permite que el paso de check termine exitosamente aunque Django devuelva error. El workflow no ejecuta tests ni migraciones. Un estado verde no confirma schema actualizado, API funcional ni assets cargados.

`main_cre-app-api.yml` tiene trigger push en `main` y manual. Instala dependencias en un runner, sube artefacto Python para despliegue a la misma app mediante login OIDC; no compila el frontend ni ejecuta `collectstatic` como `deploy.yml`. El comentario del YAML describe Oryx como supuesto/configuración de plataforma; el repositorio no verifica la configuración viva de App Service. Un push a `main` puede iniciar ambos workflows. El trigger manual permite seleccionar otra ref; inspeccionar run, commit, workflow y artefacto antes de cualquier promoción.

Secuencia operativa recomendada:

1. Confirmar commit, rama, workflow elegido, aprobaciones y que no hay otro deploy activo.
2. Crear backup de PostgreSQL y confirmar restaurabilidad; preparar migraciones compatibles y el build de frontend actual.
3. Revisar migraciones en la misma ref/commit del artefacto. Aplicarlas mediante una fase operativa aprobada y no concurrente. En App Service Linux con build Oryx, una nota del repositorio señala que el árbol ejecutable suele estar bajo `/tmp/<build-id>/` con `antenv/bin/python`; `/home/site/wwwroot` puede ser artefacto empaquetado. Es una observación operativa, no contrato de Azure: descubrir el path efectivo por SSH y confirmar que `manage.py`, settings, venv y archivo/variables de entorno correspondan al commit desplegado.
4. Solo tras confirmar path y DB, ejecutar desde directorio/venv del release efectivo `python manage.py migrate --plan` para revisar operaciones y después `python manage.py migrate --noinput` en ventana aprobada. Registrar salida y `showmigrations`; nunca migrar dos releases a la vez. `collectstatic` ya corre en `deploy.yml`; repetirlo solo si el artefacto y directorio runtime necesitan esa operación y el resultado se verifica.
5. Reiniciar la app si se modificó configuración, comprobar startup/logs y validar las rutas indicadas en checklist. Crear superusuario por procedimiento seguro (SSH interactivo es preferible; no poner password en workflow/env persistente).

Migrations manuales tras swap pueden dejar temporalmente nuevo código frente a esquema antiguo. Priorice migraciones expand-contract compatibles. App Service deployment slots y rollback de paquete pueden ayudar, pero no revierten la DB. Confirmar uso de slots/SKU y backup antes de usarlos.

PostgreSQL desde App Service: red pública vs VNet

- **Público restringido:** solo si es aprobado. Firewall permite las outbound IPs actuales del App Service, tanto `Outbound IP addresses` como `Additional outbound IP addresses`. Obtenerlas en Properties cada vez que cambie plan/escala/configuración que afecte egress y mantener inventario/auditoría. No usar IPs impresas en guías previas: son datos específicos/históricos, no configuraciones universales.
- Desmarcar “Allow public access from any Azure service within Azure” (la advertencia en `AGENTS.md`: no se limita a tu tenant/subscription; puede abrir a servicios de cualquier tenant). Preferir reglas de IP concretas o Private Endpoint.
- **Private Endpoint/VNet:** el App Service requiere VNet Integration para salida a la VNet y una subnet de integración dedicada, distinta de subnets reservadas/PE según reglas de Azure. El Private Endpoint de la DB pertenece a red privada. DNS privado enlazado a VNet y resolución del hostname de DB deben dirigir a endpoint privado; una conexión VNet sin DNS/rutas/NSG correctos no basta. Plan/tier puede tener costo. Confirme topología, subnet delegation, DNS links, peering, rutas y reglas con el equipo de red; este manual no conoce la configuración desplegada.
- La red privada de PostgreSQL sigue siendo requisito aunque frontend cambie entre SWA y App Service: cambiar endpoint web no da conectividad backend→DB.
- Confirmar TLS al DB. La configuración del repo no declara `sslmode` explícito, así que no suponga TLS verificado por una línea de documentación antigua.

Inicio, salud y pruebas de ruta

La URL es `https://<host>/healthz`; ok solo demuestra respuesta de esa vista, no conexión DB ni readiness completa. El margen de 90 s es una nota operativa del repositorio para cold starts observados, no timeout de Gunicorn ni de configuración del proveedor. Para smoke tests externos usa un límite acorde al cold start y revisa startup/logs antes de declarar caída. Verifica por separado: / como `text/html`, un asset real generado por build como `text/css` o JavaScript, `/api/auth/csrf` como JSON y una consulta autenticada a recurso que lea DB.

Para API, paths registradas no llevan slash final. Un path desconocido puede caer en el catch-all y responder HTML con 200. Si una solicitud parece exitosa, comprueba `Content-Type` y el contrato de la respuesta. `GET /healthz` tampoco sustituye esta prueba de DB.

8. Variante opcional: Azure Static Web Apps (SWA) + App Service split-origin

Es una ruta opcional de preview en el YAML presente, no el camino de push productivo del checkout. El trigger `push` está comentado; los Pull Requests hacia `azure` llaman la acción `upload` y compilan con una URL backend escrita en el workflow. El estado de SWA, el secret, dominios, control de acceso, CORS/CSRF efectivos y preview activo se desconocen. Una preview de PR no prueba el entorno productivo.

Para usar split-origin de manera deliberada:

1. Confirmar dominio público HTTPS de SWA y App Service y estrategia de cookie compatible con navegadores objetivo.
2. Compilar la SPA con `VITE_API_URL=https://<backend-real>/api`; frontend debe incluir backend API URL como contenido público.
3. En backend, `CORS_ALLOWED_ORIGINS` y `CSRF_TRUSTED_ORIGINS` exactos: origen `https://<frontend-real>` sin ruta; credenciales CORS habilitadas (el setting del código está siempre True). No wildcard.
4. Cookies cross-site pueden ser bloqueadas por políticas de privacidad del navegador aunque se configure `SameSite=None; Secure`. Backend precisa `CSRF_COOKIE_SECURE=True`, `SESSION_COOKIE_SECURE=True`, `CSRF_COOKIE_SAMESITE=None`, `SESSION_COOKIE_SAMESITE=None`; HTTPS obligatorio. La API devuelve `csrfToken` en endpoints de auth; cliente mantiene token y manda `X-CSRFToken`; validar login, refresh, mutaciones, logout en browsers reales. No exponer credenciales y tokens en logs.
5. `ALLOWED_HOSTS` contiene host del backend, no dominio de SPA. Revisar restricciones, CSP/Static Web Apps routes y health, y confirmar configuración same-origin no se aplicó inadvertidamente. Admin/login no debe exponerse por casualidad a través de SWA.
6. Asegurar GitHub secret de SWA válido y permisos `id-token: write`; workflow checked-in usa `github_id_token` además del token SWA. Estado y autorización OIDC deben verificarse en portal.

La aplicación no ofrece evidencia en repositorio sobre estado actual de SWA, dominio custom, token, política de autorización, reglas DNS, CORS efectiva o comportamiento de cookies del navegador en producción.

9. Verificación previa y posterior al despliegue

Antes

- ☐ Aprobación de cambio, commit/tag, rama, destino y workflow único confirmados.
- ☐ Diff revisado: migraciones incluidas, configuración de rutas/estáticos alineada, dependencias reproducibles.
- ☐ No hay secretos ni `.env` en artefacto; `DEBUG=False`, `SECRET_KEY` único.
- ☐ `ALLOWED_HOSTS`, orígenes CSRF/CORS y URLs de frontend coinciden con DNS real.
- ☐ DB existe, credenciales de menor privilegio, conectividad/TLS probados desde runtime de aplicación.
- ☐ Acceso de red a DB privado o firewall limitado; outbound IPs completas actuales revisadas si público.
- ☐ Backup recién generado, protegido y recuperación practicable; ventana/RPO/RTO acordados.
- ☐ `check --deploy`, tests, typecheck/build y auditoría de salida completados. No tratar `|| true` como aprobación.
- ☐ Plan de migración, compatibilidad y rollback de código/DB aprobado.

Después

- ☐ Build/deployment completados en workflow previsto; ningún workflow competidor desplaza el paquete.
- ☐ Startup sin errores, migrations verificadas, DB accesible.
- ☐ GET /healthz devuelve ok; probar también una ruta de API que lea DB.
- ☐ / sirve SPA; JS/CSS desde /api/static/ devuelve 200 y no hay errores de consola o mixed content.
- ☐ Autenticación: /api/auth/csrf, login, /api/auth/me, mutación protegida y logout; cookies flags/domain/path correctos.
- ☐ Admin responde en /api/sadmin-creapp-panel/ bajo políticas de acceso esperadas.
- ☐ HTTPS redirect/cabeceras proxy correctos; HTTP no sirve contenido sensible.
- ☐ Revisar logs, latencia, 4xx/5xx, conexiones/errores DB; informar resultado y enlazar evidencia sin secretos.

10. Operación continua, monitoreo y soporte

- Monitorear disponibilidad HTTPS, latencia, 5xx, logs Django/Gunicorn/Nginx/App Service, restart/cold start, saturación CPU/memoria y conexiones/espacio de DB. Definir alertas y escalamiento. /healthz no es DB check.
- Centralizar logs con retención, acceso limitado y redacción de tokens, cookies, SECRET_KEY, password y datos personales. No activar DEBUG para investigar en producción; reproducir en staging y usar trazas/logs seguros.
- Parchar OS/runtime/dependencias bajo control de cambios; revisar requirements.txt, package-lock.json, runtime y vulnerabilidades. Fijar actualizaciones en cambios explícitos y probar compatibilidad.
- Rotar credenciales DB/secrets, invalidar credenciales antiguas y reiniciar el proceso para cargar App Settings nuevos. Rotación de SECRET_KEY puede invalidar sesiones; coordinar.
- Ejecutar backup periódico con cifrado, retención fuera de host/entorno, verificación automática y simulacro de restore. Verificar TLS también para las utilidades CLI de backup.
- Para cada release conservar commit, artefacto/logs, schema state, backup ID, ventana, resultado de checks y operador. No almacenar valores secretos en la evidencia.

11. Diagnóstico rápido

Síntoma	Comprobar
App arranca con 500	Logs de startup/traza; DEBUG=False oculta detalle al público; ejecutar check/migrations, verificar env y build frontend entregado
400 “Invalid HTTP_HOST”	Hostname público no figura en ALLOWED_HOSTS; solo hostname exacto sin esquema
CSRF 403 same-origin	HTTPS/origen enviado, CSRF_TRUSTED_ORIGINS, cookie secure/SameSite, X-CSRFToken, cookies de sesión; revisar ruta /api/auth/csrf y payload csrfToken
Split-origin falla login/403	CORS exacto con credentials, CSRF exacto, TLS, SameSite=None + Secure, cookies enviadas y restricciones third-party del browser
API funciona pero frontend vacío	Build VITE_API_URL/VITE_STATIC_BASE, frontend/dist/index.html, collectstatic, ruta efectiva /api/static/, errores de JS y base URL. No asumir /static/
Estáticos 404	STATIC_URL del checkout es /api/static/; confirmar manifest y static root, proxy/Nginx no reescribe ruta; rebuild/recollect
Admin 404	Ruta actual /api/sadmin-creapp-panel/, no rutas históricas; verificar LOGIN_URL, urls.py, middleware proxy exemptions
DB connection refused/timeout	DNS FQDN, puerto, firewall/NSG/rutas, outbound IPs completas del App Service o VNet/private endpoint; no abrir DB a Internet
DB “does not exist”	Crear explícitamente DB POSTGRES_DB en servidor correcto; migraciones no crean base de datos
TLS DB error	Confirmar CA/certificados, hostname, cliente psycopg/CLI y requisitos de proveedor; no deshabilitar validación como solución

Respuesta 403 inesperada en API/health/admin	Si <code>PROXY_ACCESS_SECRET</code> configurado, comprobar header proxy y <code>PROXY_ACCESS_EXEMPT_PATHS</code> ; middleware devuelve 403 directo para rutas no exentas
App Service deploy verde pero código inesperado	Revisar workflows concurrentes y commit/artefacto final; ambos workflows de App Service apuntan a <code>cre-app-api</code>
Health timeout al arrancar Azure	Revisar cold start; permitir al menos 90s según nota repo, observar logs y separar proceso lento de fallo real

No cambie `DEBUG=True` en producción, no añada CORS wildcard, no publique PostgreSQL y no copie valores de IP/hostname de docs históricas para “probar rápido”.

12. Enlaces de referencia oficiales

Consultar documentación vigente del proveedor antes de ejecutar comandos porque portal, CLI y requisitos SKU cambian:

- Django: [lista de comprobación del despliegue](#) y [archivos estáticos](#).
- Servidor HTTP: [despliegue con Gunicorn](#) y [proxy de Nginx](#).
- PostgreSQL: [copias de seguridad y restauración](#) y [TLS con libpq](#).
- Azure App Service: [Python en Linux](#), [configuración](#), [integración con VNet](#) y [direcciones IP de entrada y salida](#).
- Azure Database for PostgreSQL Flexible Server: [red privada](#) y [reglas de firewall](#).
- Azure Static Web Apps: [despliegue mediante GitHub Actions](#).
- GitHub Actions: [gestión de secretos](#).

Estas URLs son referencias documentales, no confirmación de la topología ni estado de recursos de esta instalación.